

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Криптографические методы обеспечения информационной безопасности»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

«Основные структурные элементы алгоритма AES»

Выполнили:

Лим Алина Олеговна, студент группы N3349



Проверил:

Таранов Сергей Владимирович, доцент ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Введение	3
1 Ход работы	4
1.1 Эмуляция алгоритма в Cryptool 2	4
1.1.1 AES Visualization	4
1.1.2 Лавинный эффект	11
1.1.3 AES known-plaintext analysis.....	15
1.1.4 Differential Cryptanalysis Tutorial.....	15
1.2 Шифрование «вручную».....	16
1.3 Шифрование с использованием OpenSSL.....	17
Заключение.....	22

ВВЕДЕНИЕ

Цель работы: изучить основные принципы работы алгоритмы AES.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать эмуляцию алгоритма AES и примитивных атак на шифр, используя Cryptool 2. Выделить основные необходимые настройки шифра и требуемые ограничения на параметры;
- выполнение 1 цикла раундовой функции алгоритма AES вручную, то есть выполнение всех функций, входящих в раундовую функцию AES для фиксированного входного двоичного вектора с отображением промежуточных значений шифрования. Также для подробного изучения шифра может быть использована программная реализация 1 раунда (или полной системы) AES в режиме отладки с выводом промежуточных значений шифрования;
- анализ принципов использования криптосистемы в современных приложениях на примере библиотеки openssl.

1 ХОД РАБОТЫ

1.1 Эмуляция алгоритма в Cryptool 2

1.1.1 AES Visualization

Зашифруем текст используя AES Visualization.

Исходный текст (HEX): 9D1FD2501AD1D42468A36785D131911F

Ключ (HEX): 3768533C4D20D24A16A2813C633426B1

AES начинается с этапа "Key Expansion", в котором раундовые ключи являются производными от исходного ключа. После расширения проходит начальный раунд, в ходе которого исходный ключ применяется к открытому тексту. В зависимости от длины ключа алгоритм будет проходить через 9, 11 или 13 раундов. Каждый раунд состоит из четырех операций: «SubBytes», «ShiftRow», «MixColumns» и «AddRoundKey». Как только эти раунды завершены, выполняется последний раунд, в котором нет шага «MixColumn».

1.1.1.1 Key Expansion

16 байт встает на место 13, а 13–15 сдвигаются на один вниз (рисунок 1).

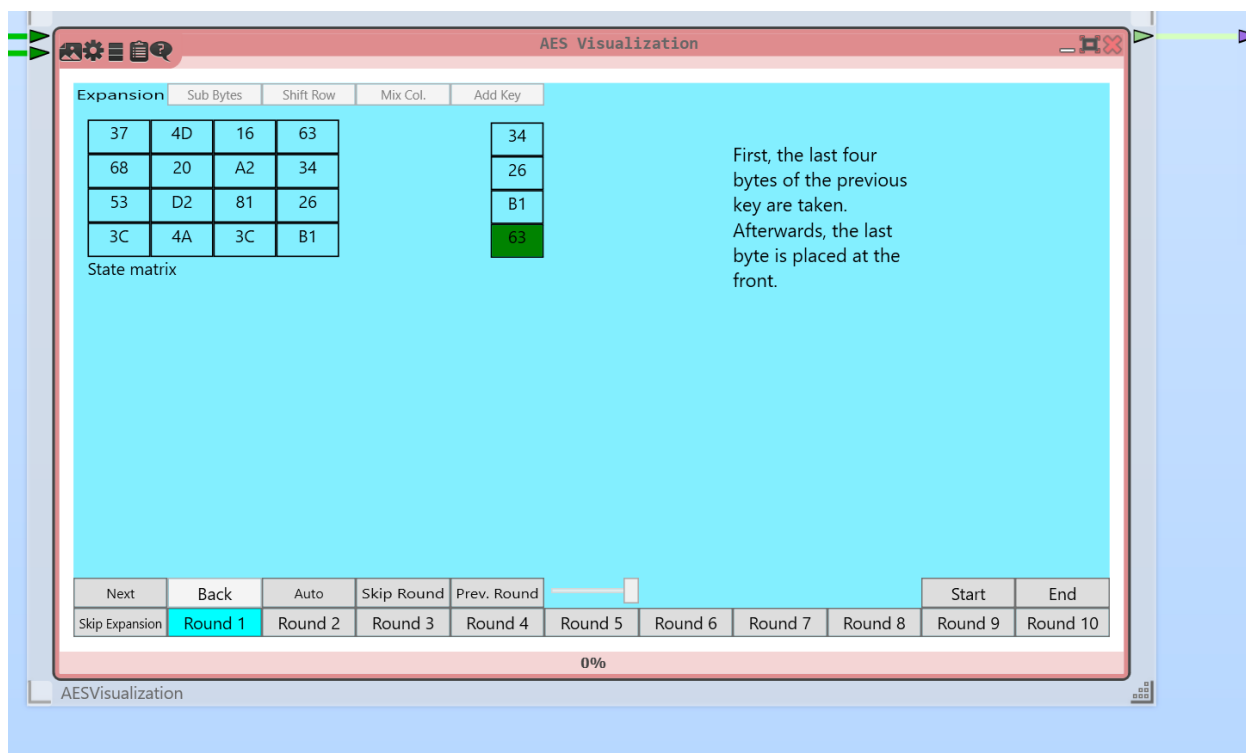


Рисунок 1 – Key Expansion (1)

Каждый байт делится пополам на 4 бита и в «S-Box» берется их пересечение (рисунок 2).

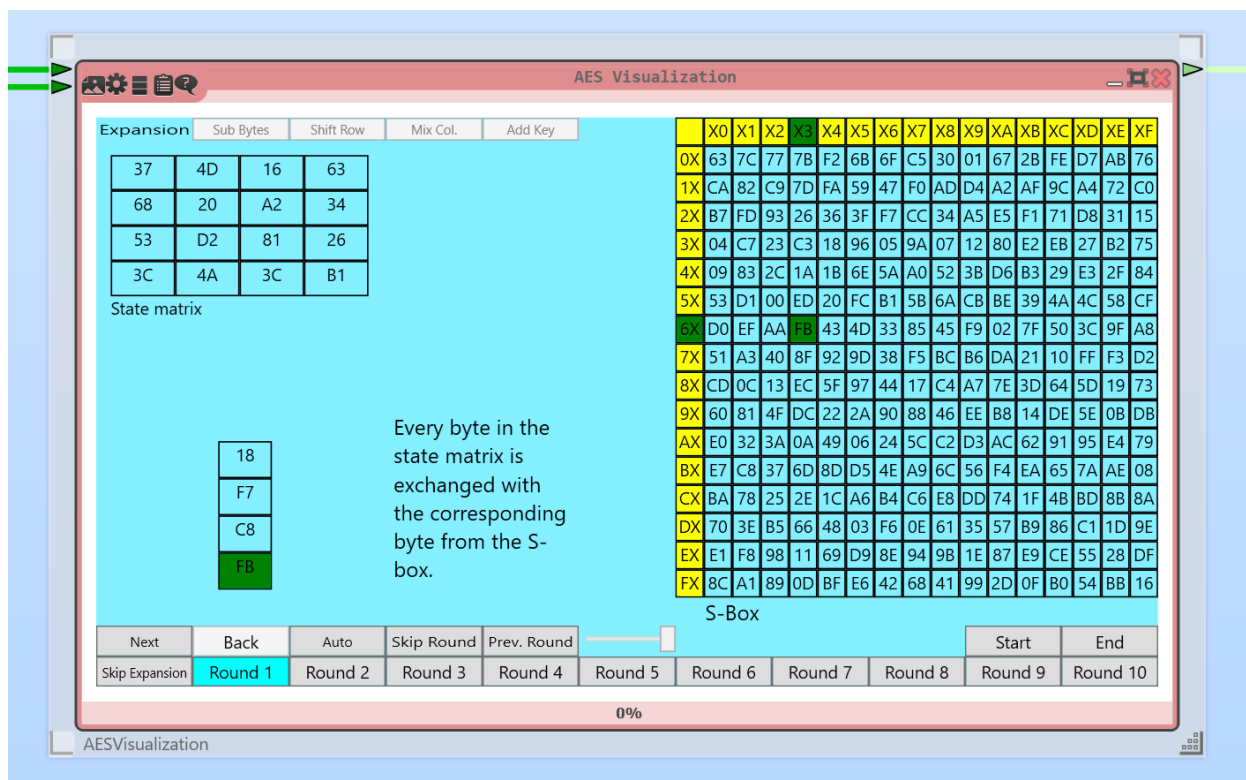


Рисунок 2 – Key Expansion (2)

Далее выполняется XOR 4 байт и столбца, у которого в первой ячейке указывается номер соответствующего раунда (раундовая постоянная) (рисунок 3).

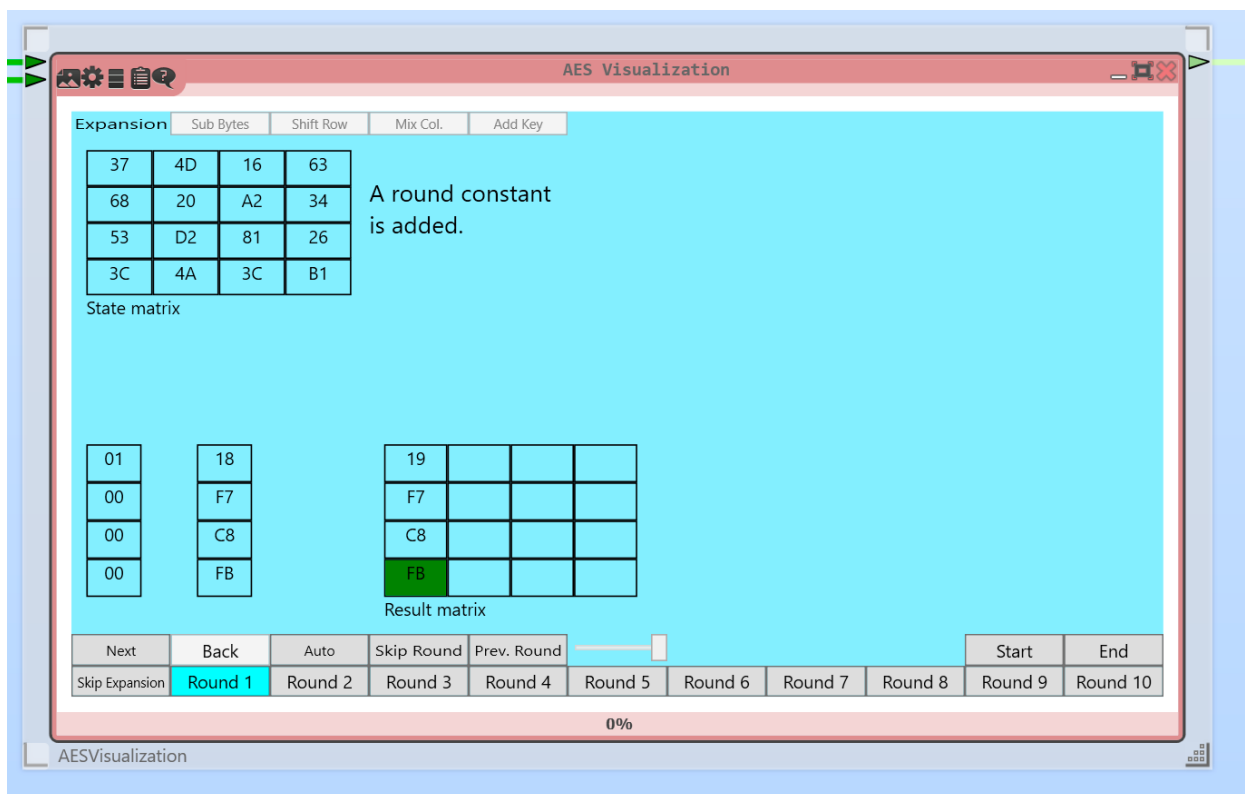


Рисунок 3 – Key Expansion (3)

1 столбец матрицы состояний XOR с ключевым столбцом. Последующие столбцы матрицы состояний XOR с результатами предыдущего XOR (рисунок 4).

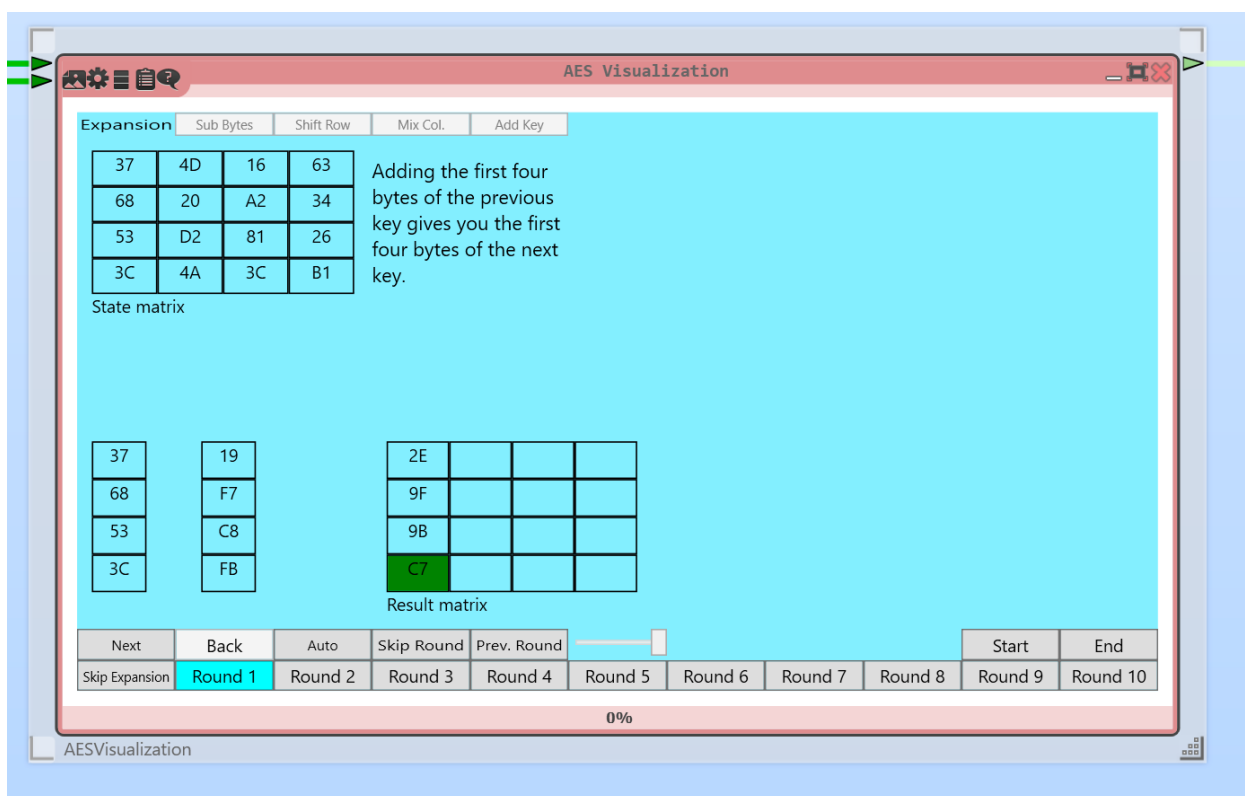


Рисунок 4 – Key Expansion (4)

Получили матрицу раундового ключа для 1 раунда. Она будет использоваться для генерации матрицы ключа для 2 раунда (рисунок 5).

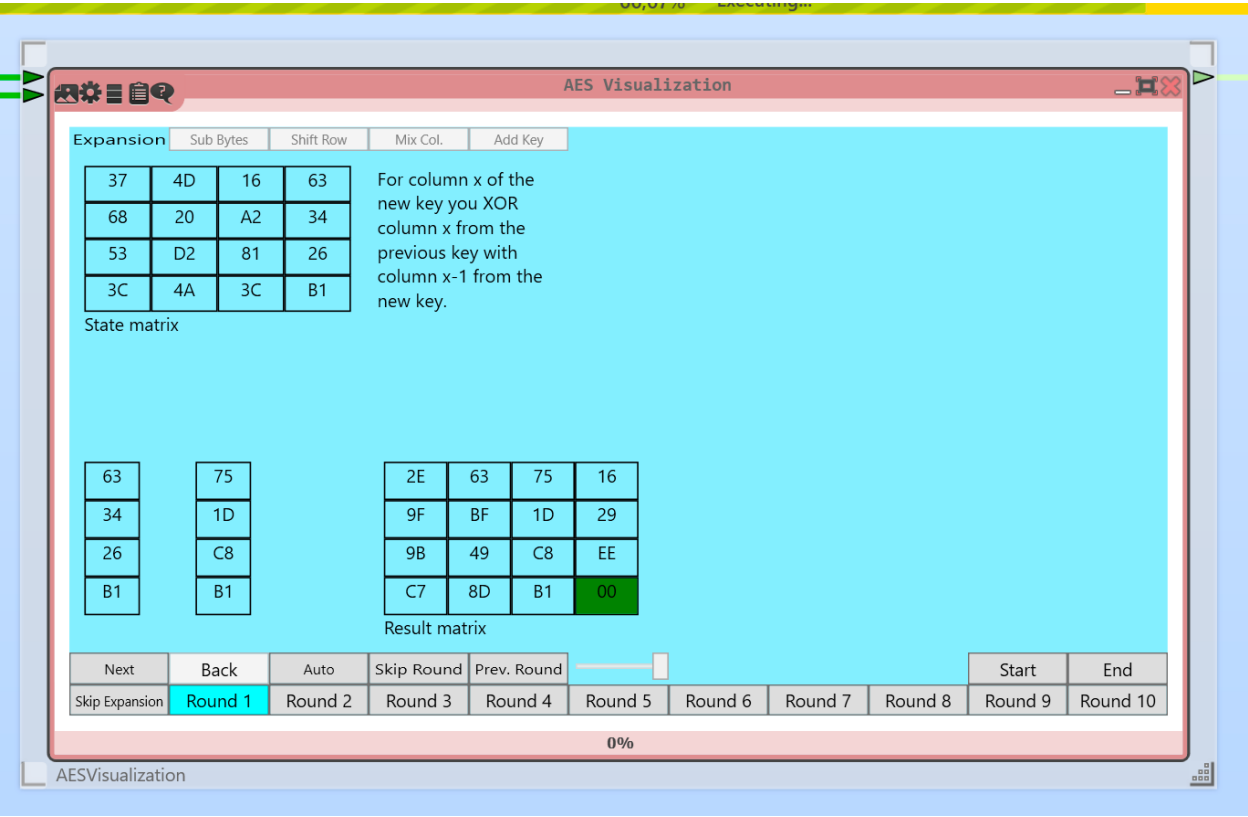


Рисунок 5 – Key Expansion (5)

1.1.1.2 Encryption

XOR начальных открытого текста и ключа (рисунок 6).

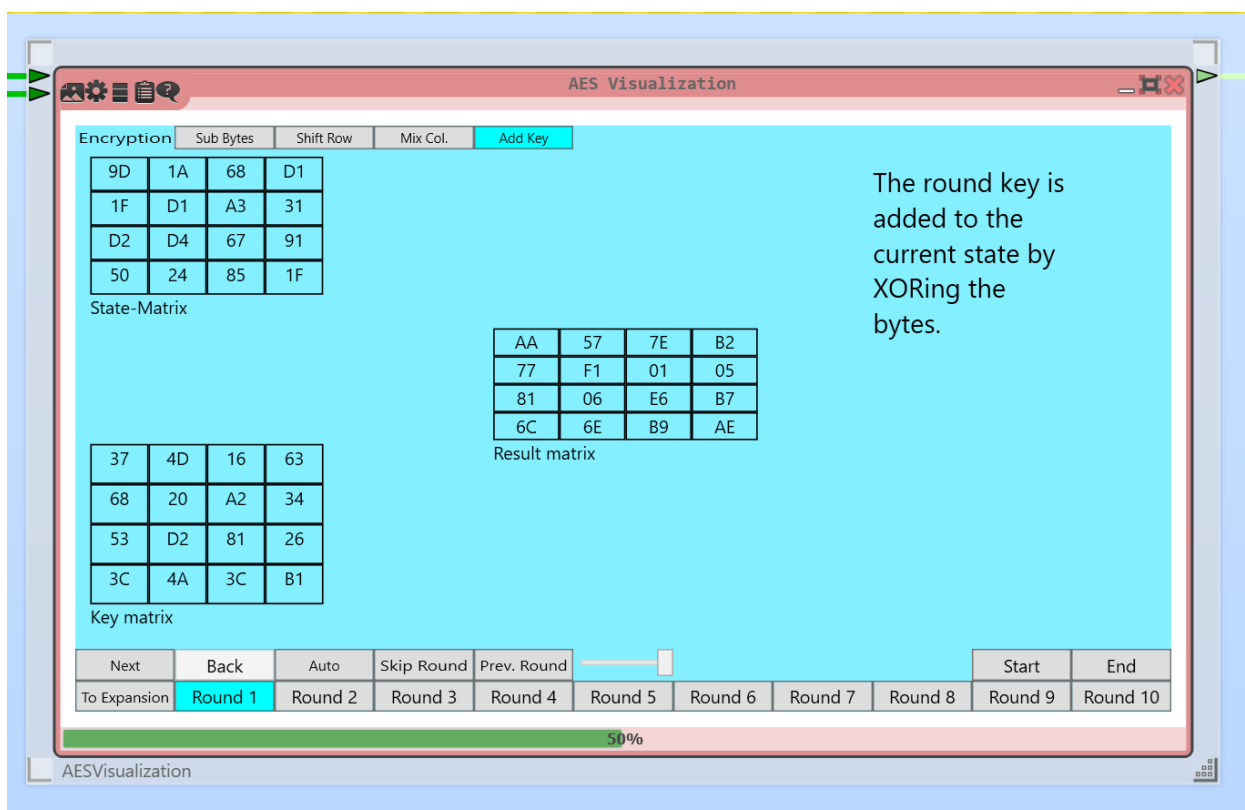


Рисунок 6 – Encryption

Каждый байт, полученный матрицы делится пополам по 4 бита и в «S-Box» берется их пересечение (рисунок 7).

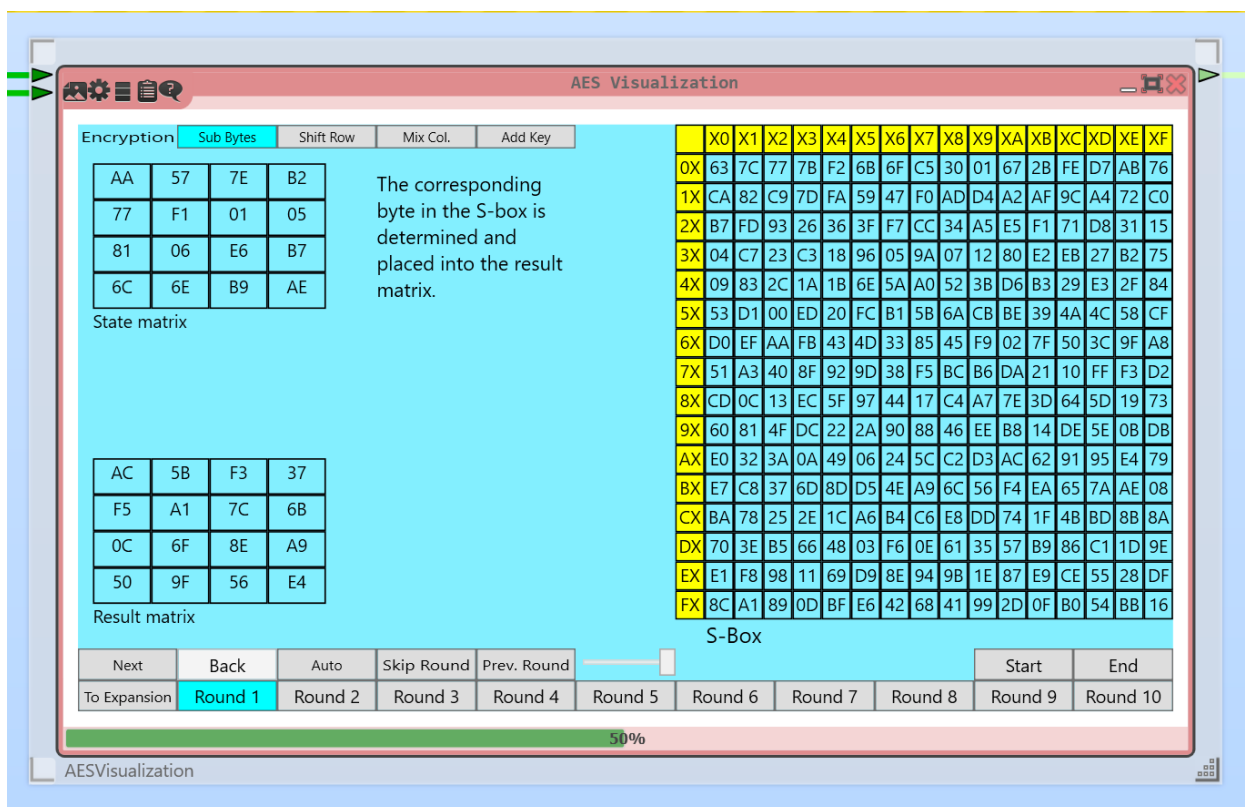


Рисунок 7 – Sub Bytes

Происходит сдвиг строки, каждая строка сдвигается на число, соответствующее ее номеру (0–4) (рисунок 8).

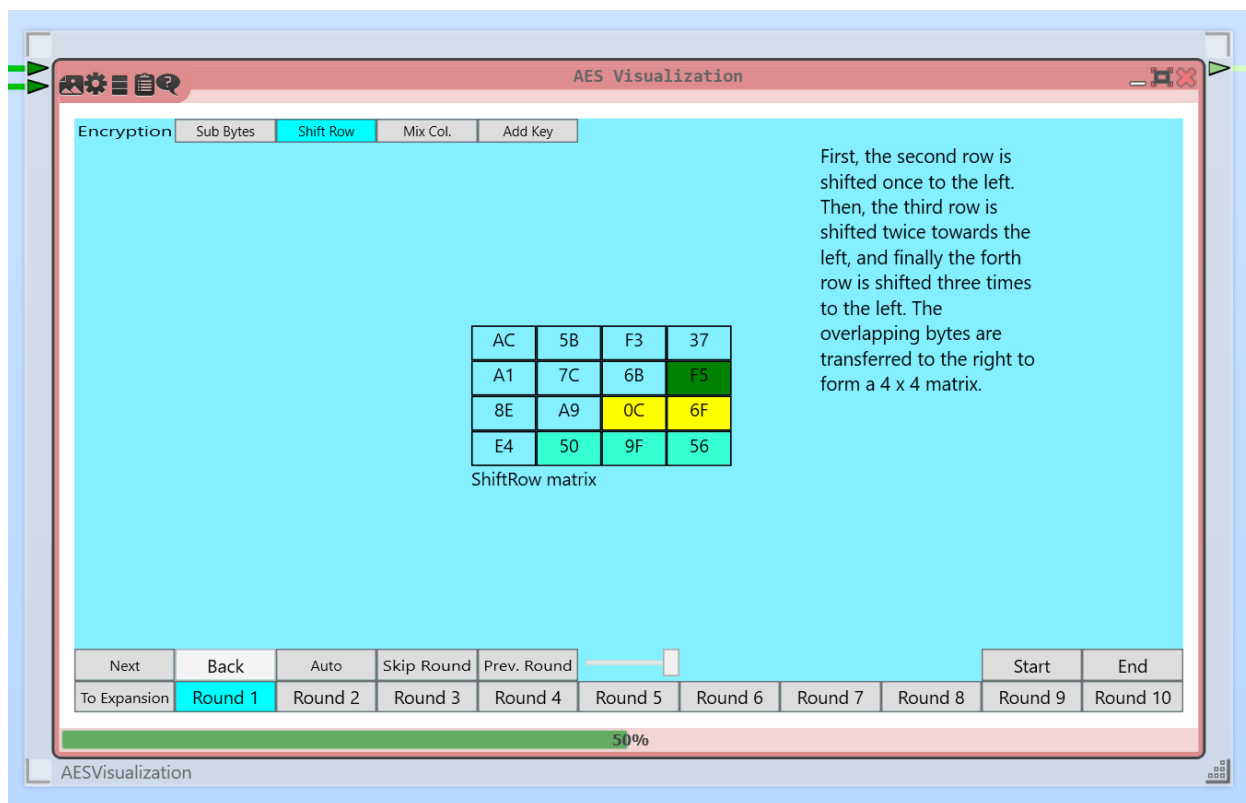


Рисунок 8 – Shift Row

Происходит перемешивание столбцов, перемножаются столбцы матрицы состояний и строки мультипликативной матрицы (рисунки 9, 10).

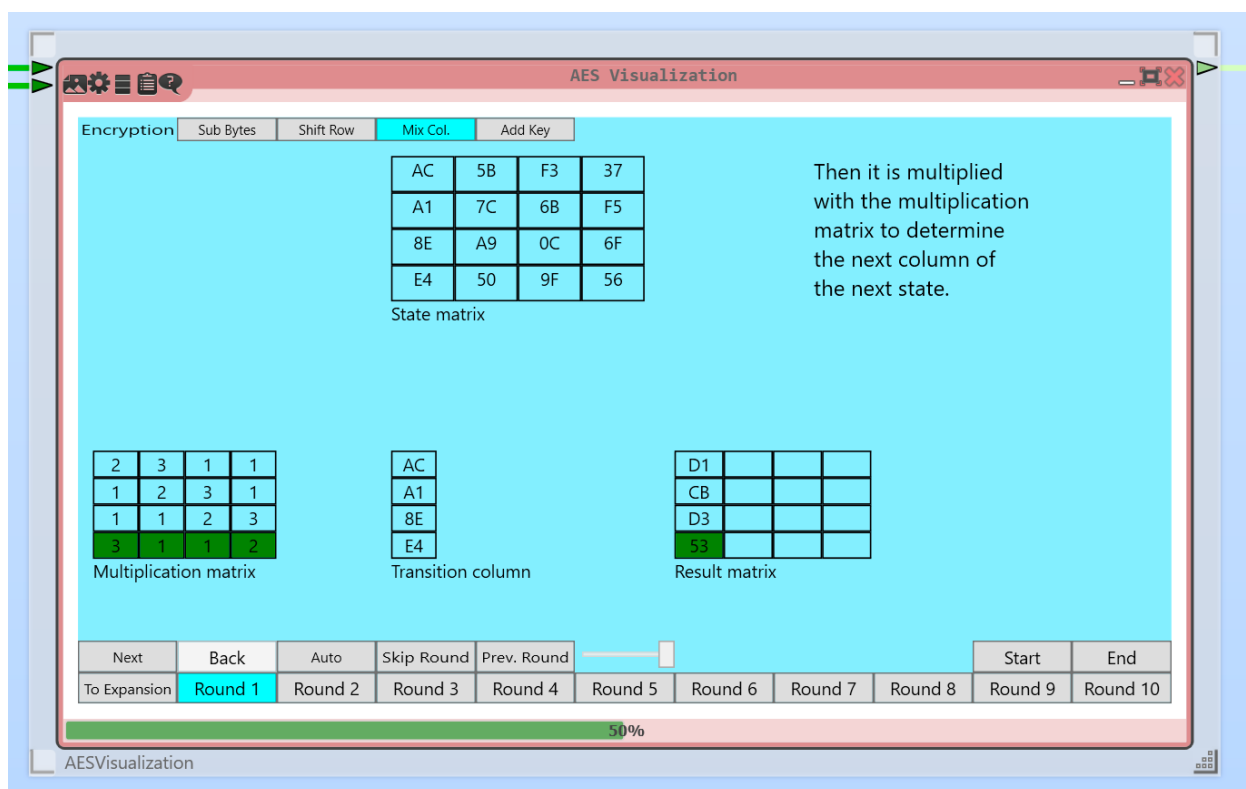


Рисунок 9 – Mix Col. (1)

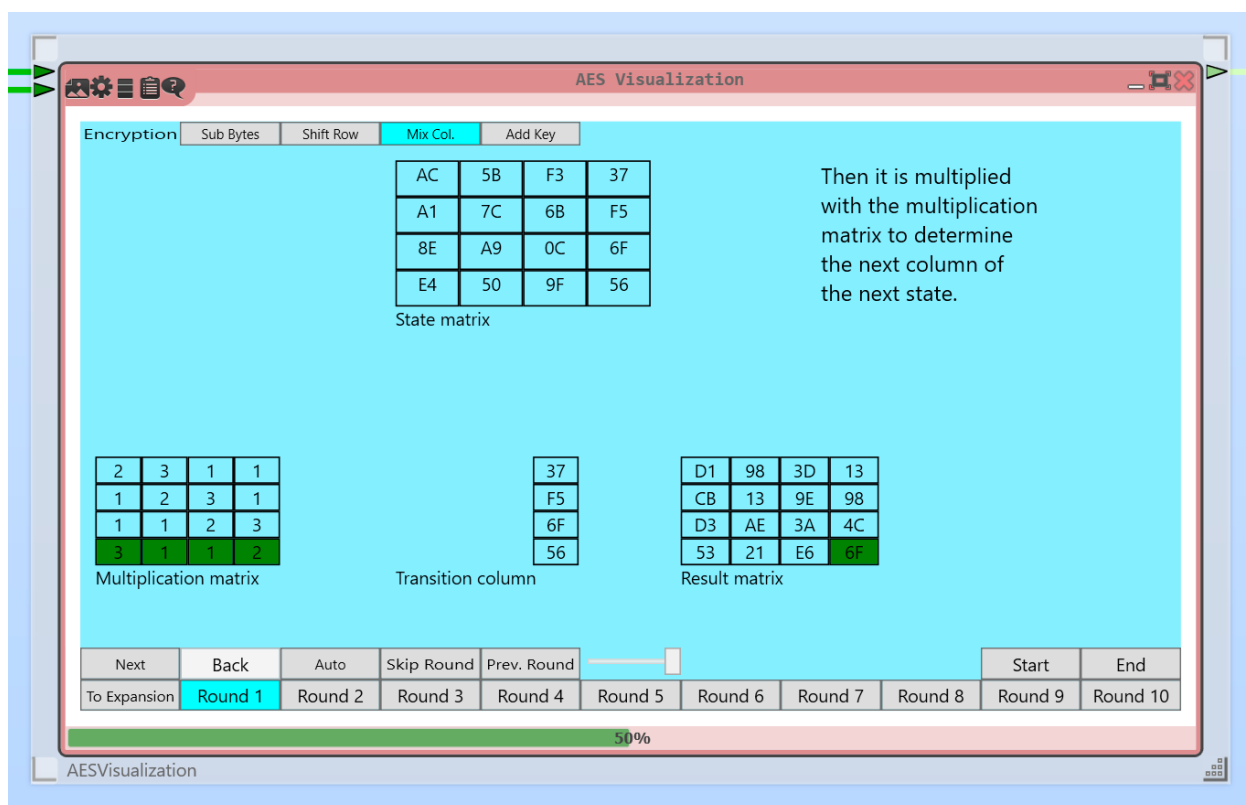


Рисунок 10 – Mix Col. (2)

Выполняется XOR ячеек матрицы состояний и полученной матрицы раундового ключа (рисунок 11).

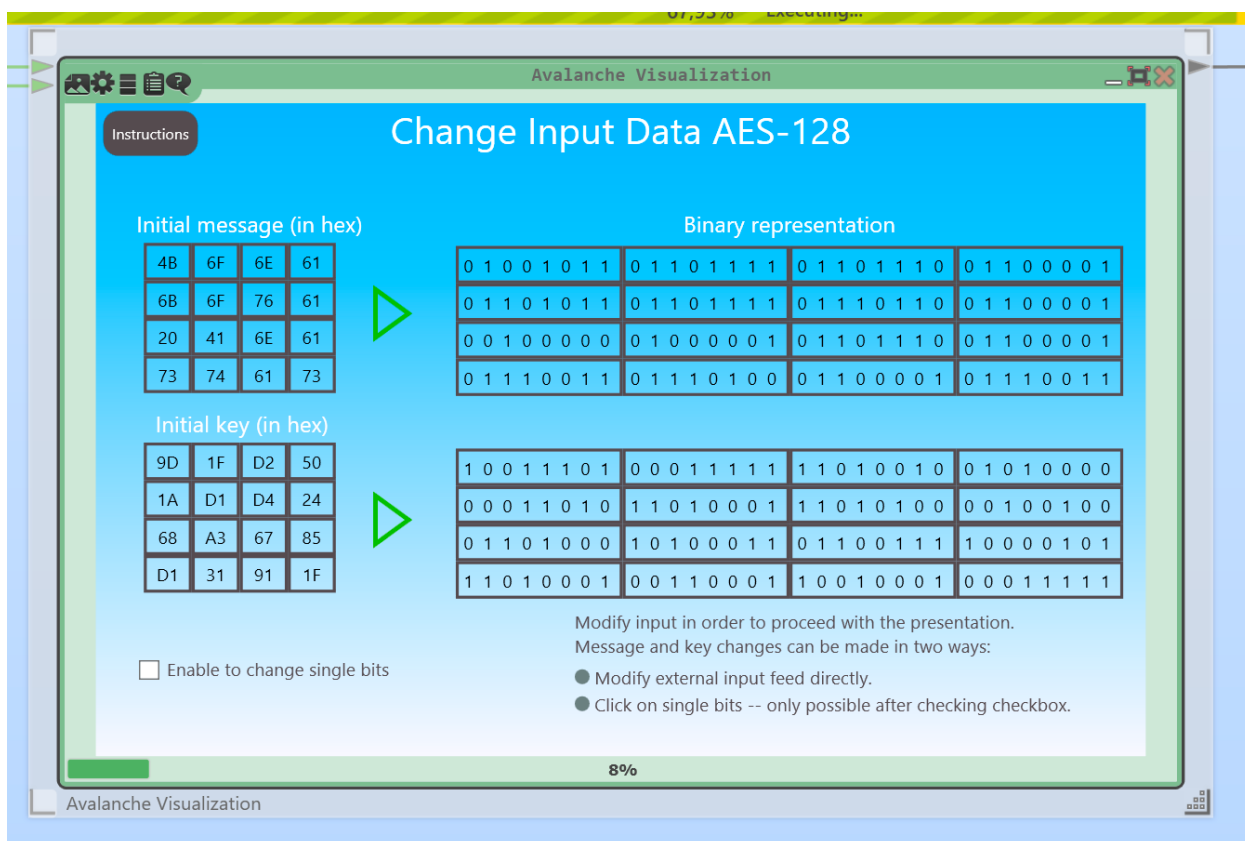


Рисунок 13 – Avalanche Visualization

Искажем 1 бит исходного текста (рисунок 14).

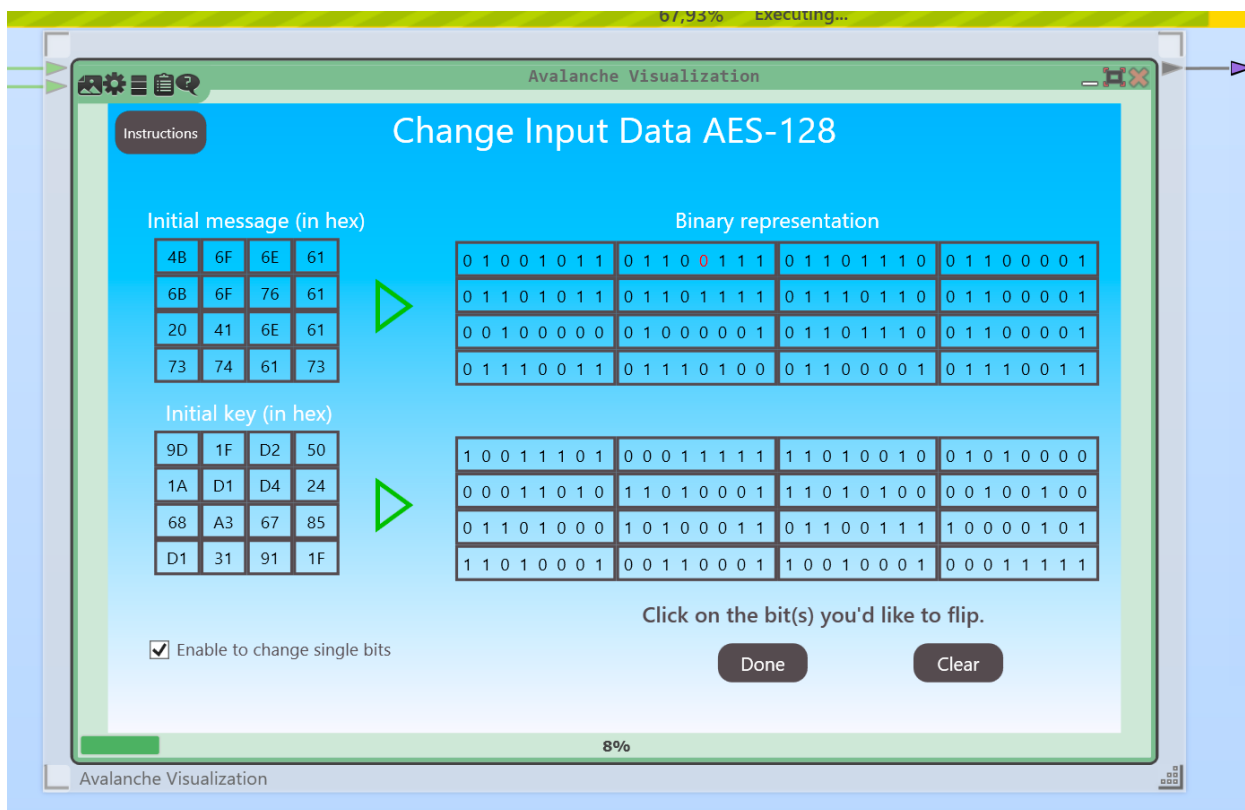


Рисунок 14 – Искажение 1 бит

При искажении 1 бит открытого текста уже на 3 раунде процент искаженных битов составляет 46,1% (рисунок 15).

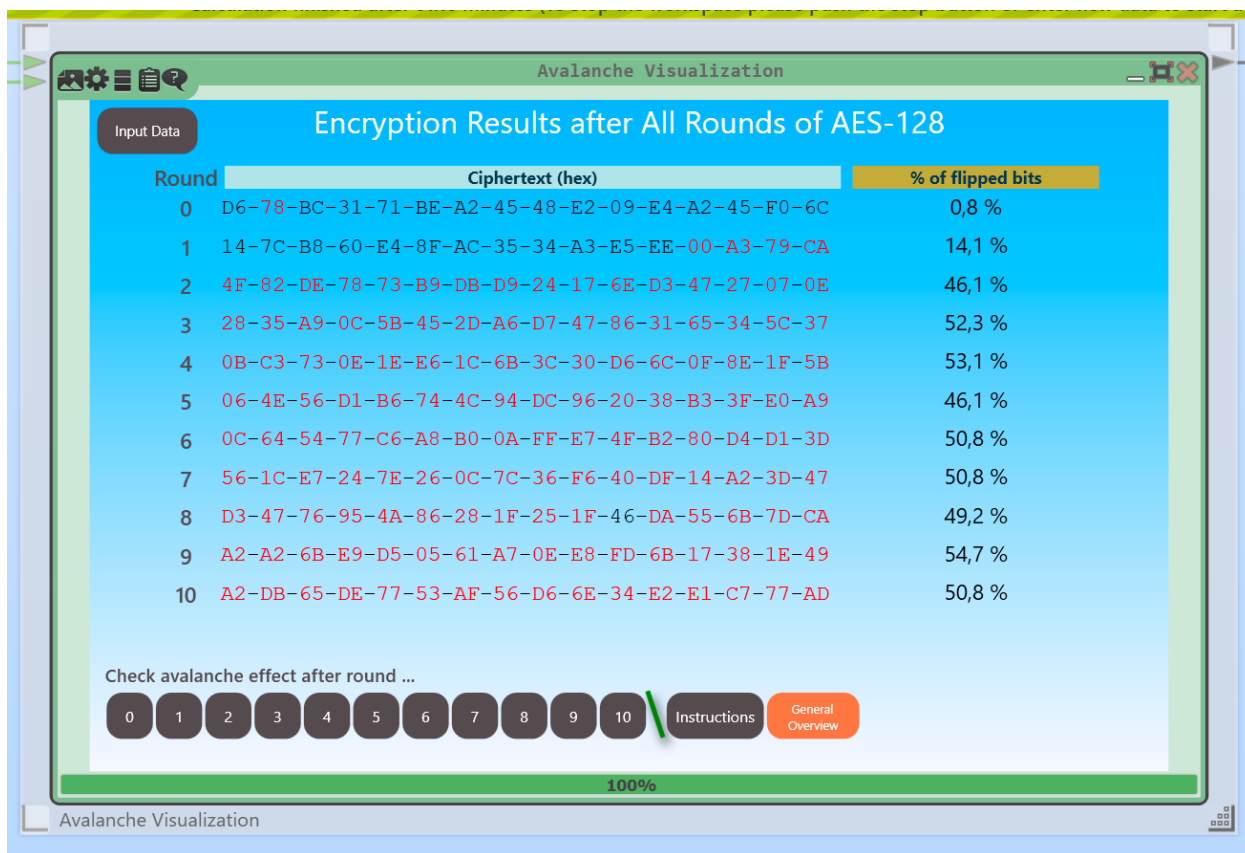


Рисунок 15 – Результат

Исказим дополнительно 1 бит текста и 2 бита ключа (рисунок 16).

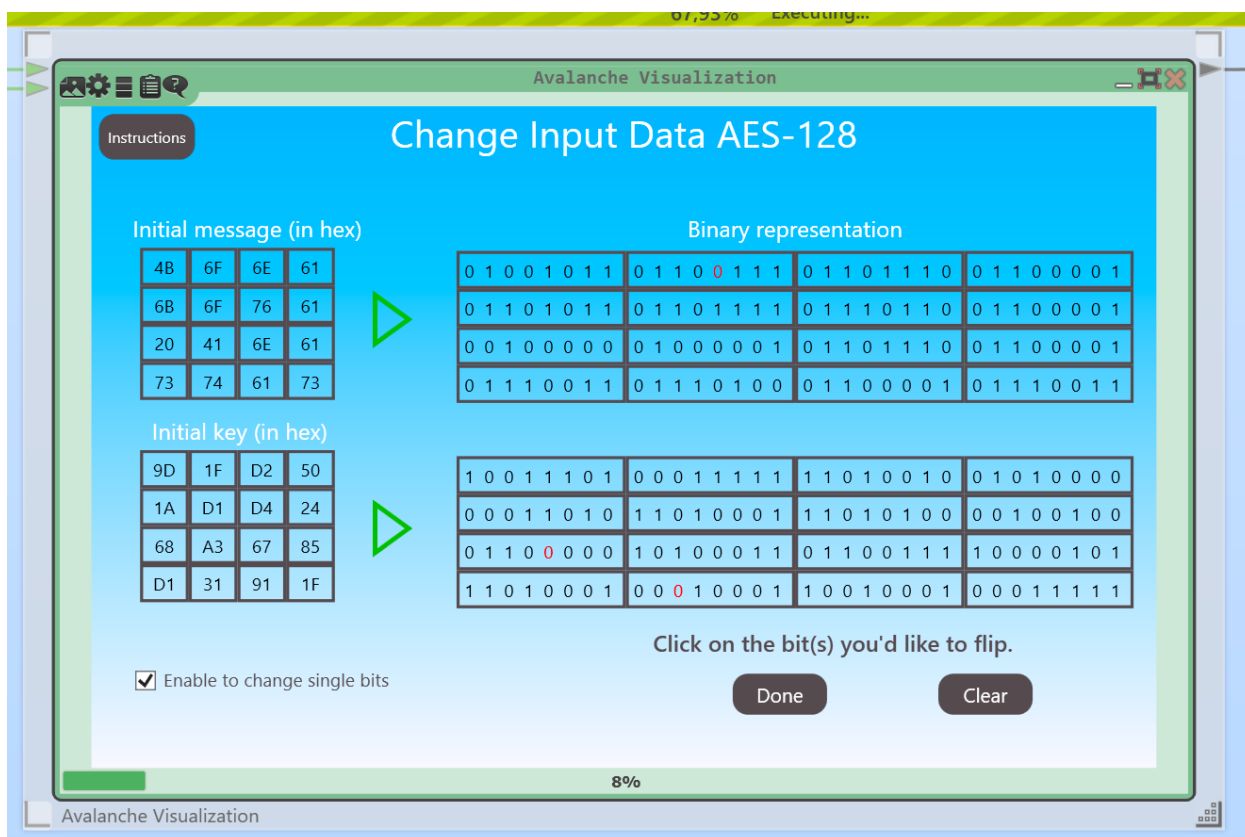


Рисунок 16 – Дополнительные искажения

Результат представлен на рисунке 17.

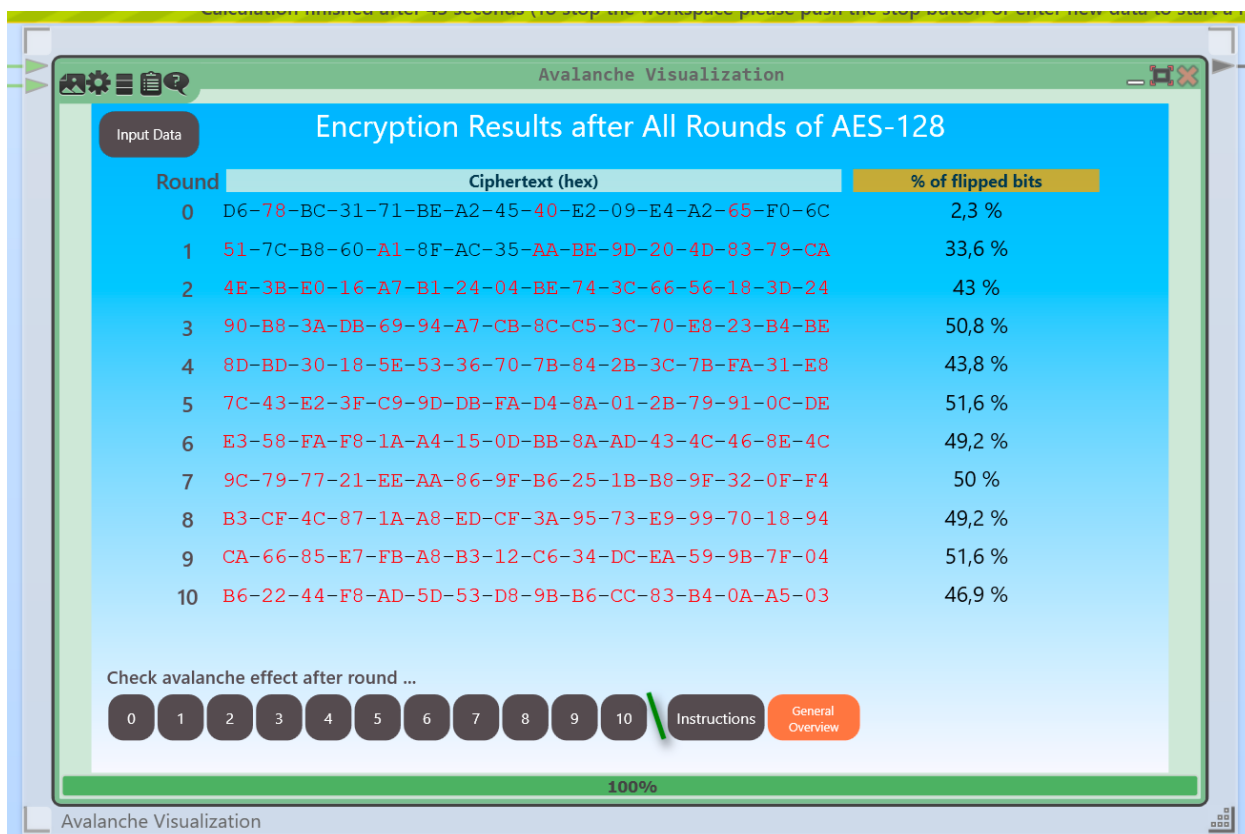


Рисунок 17 – Конечный результат

1.1.3 AES known-plaintext analysis

Проведем атаку AES known-plaintext analysis при известной половине ключа (рисунок 18).

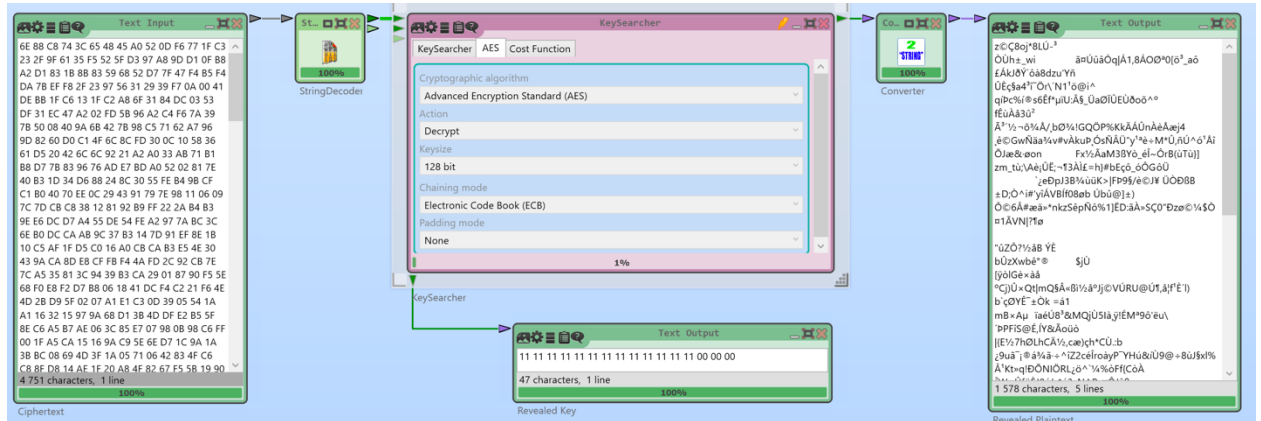


Рисунок 18 – Атака на шифр

Будем постепенно увеличивать количество известных байт. Текст успешно дешифруется при известных 14/16 байт ключа (рисунок 19).

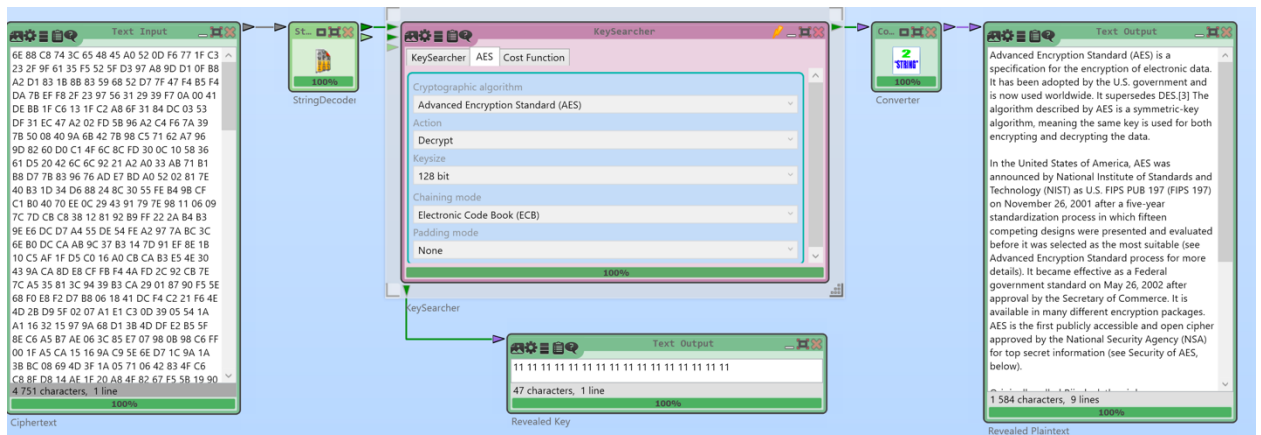


Рисунок 19 – Дешифрованный текст

1.1.4 Differential Cryptanalysis Tutorial

Дифференциальный криптоанализ (ДКА) — это вероятностный метод анализа симметричных блочных шифров. С помощью этой атаки пытаются итеративно восстановить подразделы. Основным элементом ДКА являются пары блоков зашифрованного текста, открытый текст которых имеет определенное различие. Делается попытка определить поток блоков через шифр на основе вероятностей и, таким образом, определить дополнительные ключи.

На каждом раунде входной блок делится на 8 байт, каждый из которых проходит через функцию подстановки S-Box. После этого данные перемешиваются с 64 битным подключом Subkey. Функция перемешивания представляет собой обычный XOR. Работа шаблона представлена на рисунке 20.

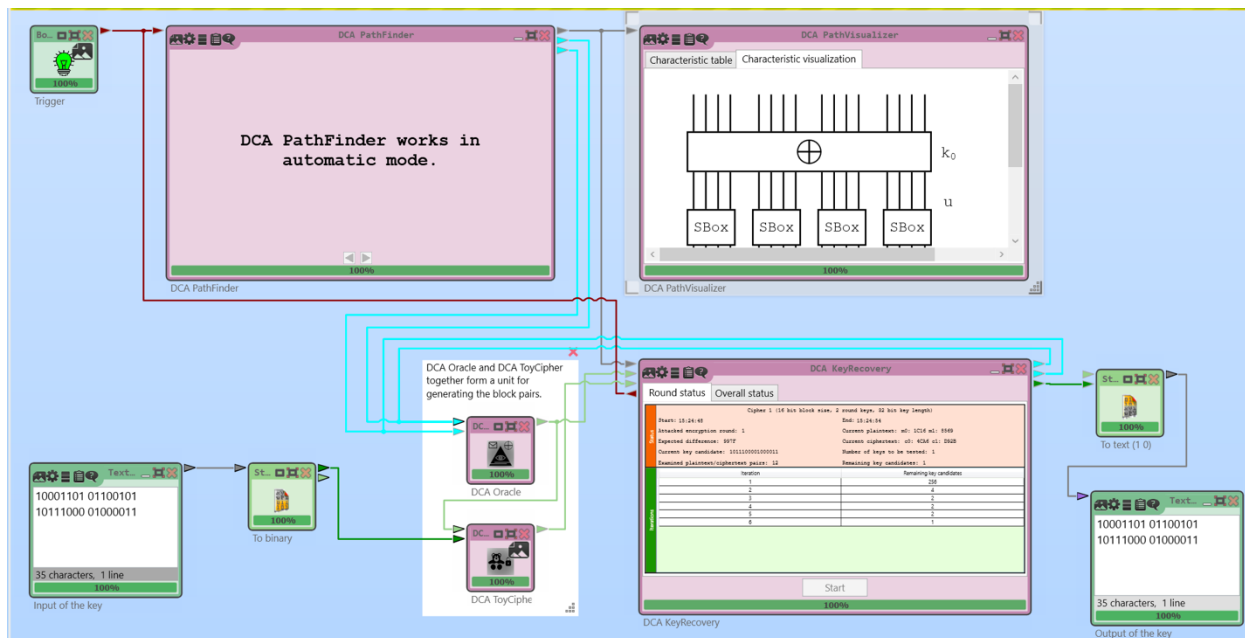


Рисунок 20 – Differential Cryptanalysis Tutorial

Для двух заранее подобранных шифротекстов $P1$ и $P2$ злоумышленник вычисляется «дифференциал» $\Delta P = P1 \oplus P2$. И с помощью ΔP пытается определить каким должен быть «дифференциал» шифротекстов $\Delta C = C1 \oplus C2$. Зачастую невозможно предугадать со 100% точностью какое именно будет иметь значение ΔC . Единственное, что может злоумышленник, это определить с какой частотой шифр возвращает различные значения ΔC , для заданного заранее ΔP . Это знание позволяет атакующему вскрыть часть ключа или ключ целиком.

1.2 Шифрование «вручную»

Выполним 1 цикл раундовой функции алгоритма AES вручную (рисунки 21, 22). Ключ и сообщения взяты случайны.

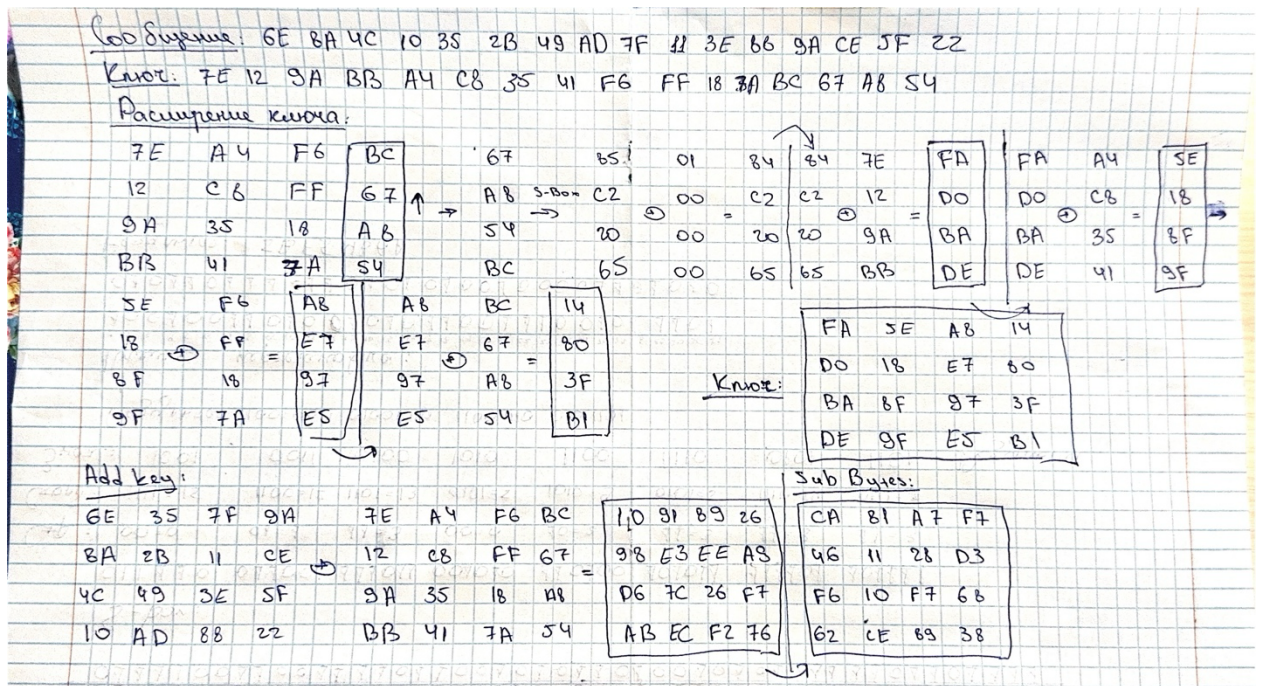


Рисунок 21 – Шифрование «вручную» (1)

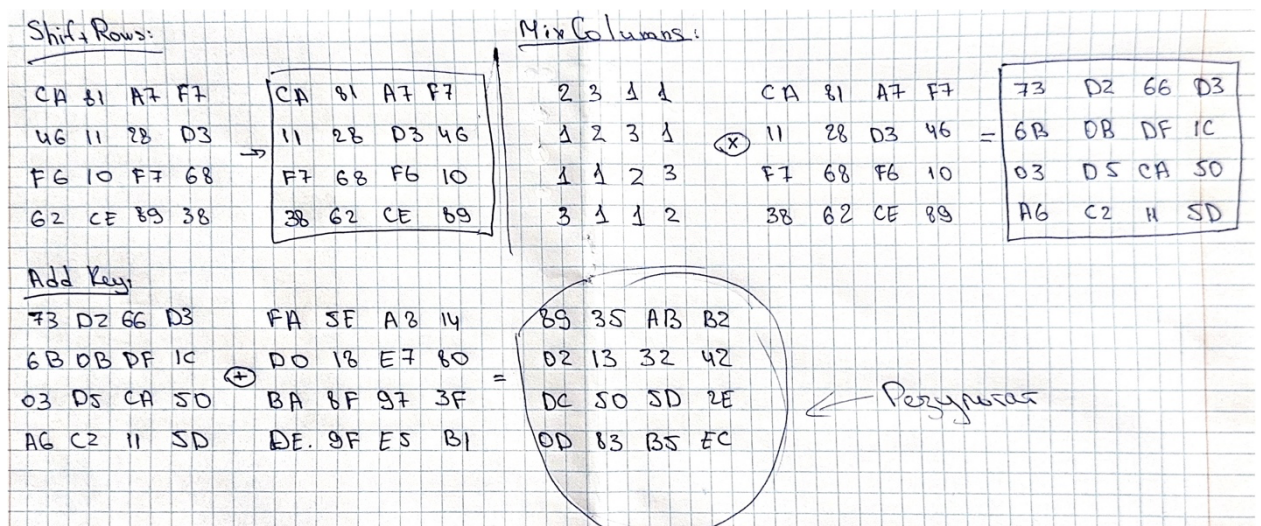


Рисунок 22 – Шифрование «вручную» (2)

1.3 Шифрование с использованием OpenSSL

Проведем шифрование текстового файла с использованием библиотеки OpenSSL. Процесс шифрования, а также используемые при этом режимы, представлены на рисунках 23–36.

```

options: bn(64,64)
compiler: cl /Z7 /Fdossl_static.pdb /Gs0 /GF /Gy /MD /W3 /wd4090 /nologo /O2 -DL_ENDIAN -DOPENSSL_PIC -D"OPENSSL_BUILDI
NG_OPENSSL" -D"OPENSSL_SYS_WIN32" -D"WIN32_LEAN_AND_MEAN" -D"UNICODE" -D"_UNICODE" -D"_CRT_SECURE_NO_DEPRECATED" -D"_WINS
OCK_DEPRECATED_NO_WARNINGS" -D"NDEBUG" -D_WINSOCK_DEPRECATED_NO_WARNINGS -D_WIN32_WINNT=0x0502
OPENSSLDIR: "C:\Program Files\Common Files\SSL"
ENGINESDIR: "C:\Program Files\OpenSSL\lib\engines-3"
MODULESDIR: "C:\Program Files\OpenSSL\lib\ossl-modules"
Seeding source: os-specific
CPUINFO: OPENSSL_ia32cap=0xffda32034f8bffff:0xd09f07ab

```

Рисунок 23 – Установленная библиотека

```

C:\Users\alinalimm>openssl enc -list
Supported ciphers:
-aes-128-cbc          -aes-128-cfb          -aes-128-cfb1
-aes-128-cfb8         -aes-128-ctr          -aes-128-ecb
-aes-128-ofb          -aes-192-cbc          -aes-192-cfb
-aes-192-cfb1         -aes-192-cfb8         -aes-192-ctr
-aes-192-ecb          -aes-192-ofb          -aes-256-cbc
-aes-256-cfb          -aes-256-cfb1         -aes-256-cfb8
-aes-256-ctr          -aes-256-ecb          -aes-256-ofb
-aes128               -aes128-wrap          -aes128-wrap-pad
-aes192               -aes192-wrap          -aes192-wrap-pad
-aes256               -aes256-wrap          -aes256-wrap-pad
-aria-128-cbc         -aria-128-cfb         -aria-128-cfb1
-aria-128-cfb8        -aria-128-ctr         -aria-128-ecb
-aria-128-ofb         -aria-192-cbc         -aria-192-cfb
-aria-192-cfb1        -aria-192-cfb8        -aria-192-ctr
-aria-192-ecb         -aria-192-ofb         -aria-256-cbc
-aria-256-cfb         -aria-256-cfb1        -aria-256-cfb8
-aria-256-ctr         -aria-256-ecb         -aria-256-ofb
-aria128              -aria192              -aria256
-bf                   -bf-cbc               -bf-cfb
-bf-ecb              -bf-ofb               -blowfish
-camellia-128-cbc     -camellia-128-cfb     -camellia-128-cfb1
-camellia-128-cfb8    -camellia-128-ctr     -camellia-128-ecb
-camellia-128-ofb     -camellia-192-cbc     -camellia-192-cfb
-camellia-192-cfb1    -camellia-192-cfb8    -camellia-192-ctr
-camellia-192-ecb     -camellia-192-ofb     -camellia-256-cbc
-camellia-256-cfb     -camellia-256-cfb1    -camellia-256-cfb8
-camellia-256-ctr     -camellia-256-ecb     -camellia-256-ofb
-camellia128          -camellia192          -camellia256
-cast                 -cast-cbc              -cast5-cbc
-cast5-cfb           -cast5-ecb             -cast5-ofb
-chacha20             -des                   -des-cbc
-des-cfb              -des-cfb1              -des-cfb8
-des-ecb              -des-edc               -des-edc-cbc
-des-edc-cfb          -des-edc-ecb           -des-edc-ofb
-des-edc3             -des-edc3-cbc          -des-edc3-cfb
-des-edc3-cfb1        -des-edc3-cfb8        -des-edc3-ecb
-des-edc3-ofb         -des-ofb               -des3
-des3-wrap            -desx                  -desx-cbc
-id-aes128-wrap        -id-aes128-wrap-pad    -id-aes192-wrap
-id-aes192-wrap-pad    -id-aes256-wrap        -id-aes256-wrap-pad
-id-smime-alg-CMS3DESwrap -idea                  -idea-cbc
-idea-cfb             -idea-ecb              -idea-ofb
-rc2                  -rc2-128               -rc2-40
-rc2-40-cbc           -rc2-64                -rc2-64-cbc
-rc2-cbc              -rc2-cfb               -rc2-ecb
-rc2-ofb              -rc4                   -rc4-40
-seed                 -seed-cbc              -seed-cfb
-seed-ecb             -seed-ofb              -sm4
-sm4-cbc              -sm4-cfb               -sm4-ctr
-sm4-ecb              -sm4-ofb

```

Рисунок 24 – Поддерживаемые алгоритмы

```
C:\Users\alinalimmm>openssl enc -aes-128-cbc -e -in Desktop/text.txt -out text1.txt.enc
enter AES-128-CBC encryption password:
Verifying - enter AES-128-CBC encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

Рисунок 25 – Шифрование AES-128-CBC

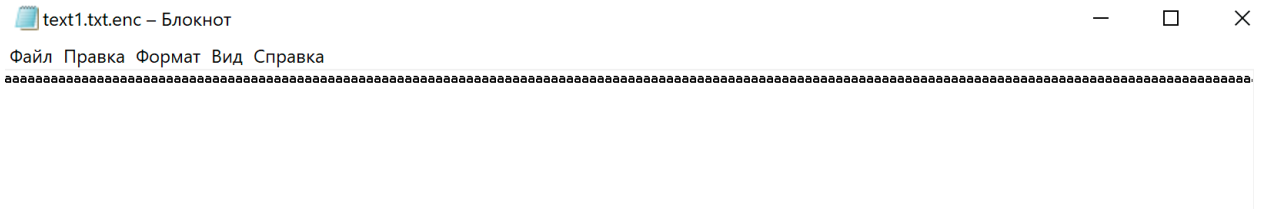


Рисунок 26 – Зашифрованный файл

```
C:\Users\alinalimmm>openssl enc -aes-128-cbc -d -in text1.txt.enc -out text2.txt
enter AES-128-CBC decryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

Рисунок 27 – Расшифровка файла

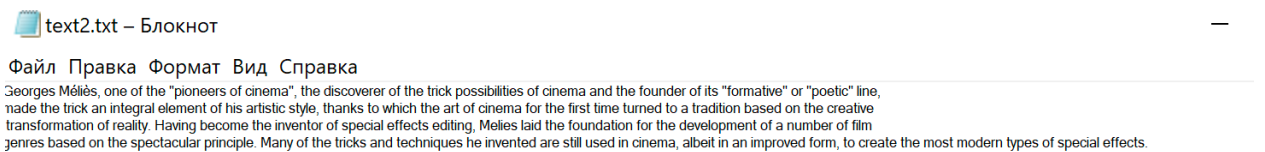


Рисунок 28 – Расшифрованный файл

```
C:\Users\alinalimmm>openssl enc -aes-128-ofb -e -in Desktop/text.txt -out text2.txt.enc
enter AES-128-OFB encryption password:
Verifying - enter AES-128-OFB encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

Рисунок 29 – Шифрование AES-128-OFB

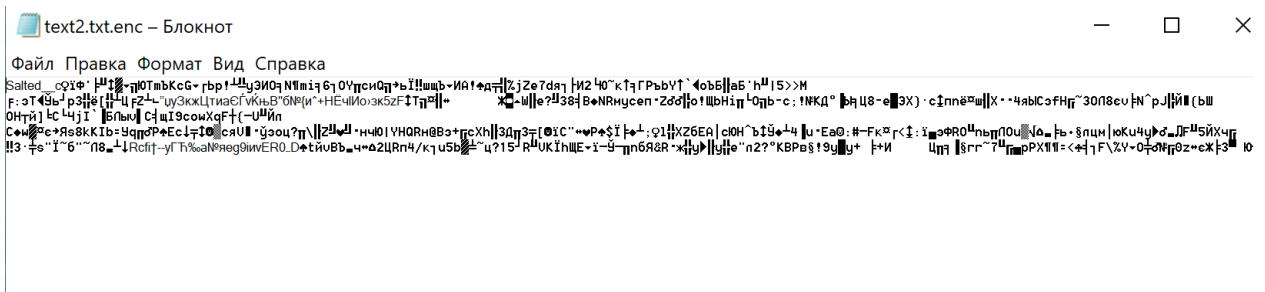


Рисунок 30 – Зашифрованный файл


```
C:\Users\alinalimmm>openssl enc -aes-128-ofb -d -in text2.txt.enc -out text3.txt
enter AES-128-OFB decryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

Рисунок 31 – Расшифровка файла

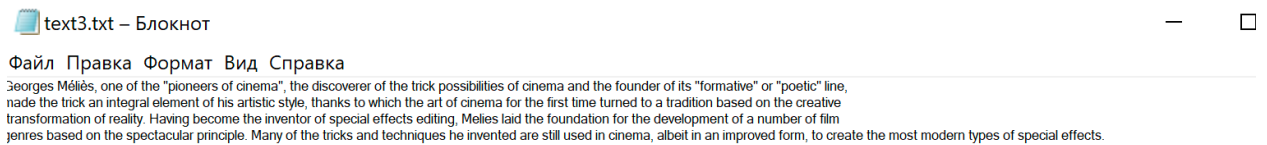


Рисунок 32 – Расшифрованный файл

```
C:\Users\alinalimmm>openssl enc -aes-128-ecb -e -in Desktop/text.txt -out text3.txt.enc
enter AES-128-ECB encryption password:
Verifying - enter AES-128-ECB encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

Рисунок 33 – Шифрование AES-128-ECB

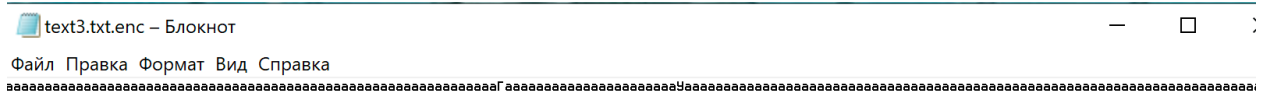


Рисунок 34 – Зашифрованный файл

```
C:\Users\alinalimmm>openssl enc -aes-128-ecb -d -in text3.txt.enc -out text4.txt
enter AES-128-ECB decryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

Рисунок 35 – Расшифровка файла

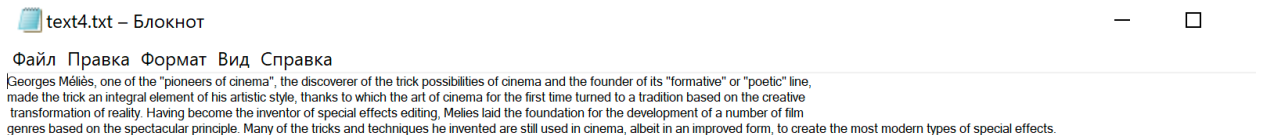


Рисунок 36 – Расшифрованный файл

Флаги и дополнительные параметры шифрования AES в OpenSSL:

- -salt – добавлять соль в шифротекст;
- -a – кодирование/декодирование Base64, в зависимости от флага шифрования;
- -k – ввод ключа;
- -iter – задание числа итераций pbkdf2.

ЗАКЛЮЧЕНИЕ

В ходе работы был проведен анализ эмуляции алгоритма AES и примитивных атак на шифр с использованием CgypTool 2. Были выделены основные необходимые настройки шифра и требуемые ограничения на параметры. После этого был вручную выполнен 1 цикл раундовой функции алгоритма DES. Также были проанализированы принципы использования криптосистемы в современных приложениях на примере библиотеки OpenSSL. Таким образом, поставленные задачи были выполнены, цель достигнута.